Digital System Design

Module 1 — Number Systems, Boolean Algebra, and Combinational Logic

Course Code: EC302 | **Semester:** 3rd Semester (ECE)

MAKAUT B.Tech Maulana Abul Kalam Azad University of Technology

Author: Biraj Sarkar | Cooch Behar Government Engineering College

Comprehensive, exam-ready lecture notes covering Number Systems, Signed Arithmetic, Boolean Algebra, Canonical Forms, Karnaugh Maps, Binary Codes, and MSI Combinational Devices.

© Biraj Sarkar | Academic Non-Commercial Notes | CC BY-NC-SA 4.0

Contents

1 Number Systems and Binary Arithmetic . . . 2
- 1.1 Radix Conversions . . . 2
- 1.2 Representation of Signed Numbers . . . 2
- 1.3 2's Complement Arithmetic and Overflow Detection . . . 3

2 Boolean Algebra and Logic Simplification . . . 3
- 2.1 Fundamental Laws of Boolean Algebra . . . 4
- 2.2 De Morgan's Theorems and the Principle of Duality . . . 4
- 2.3 Canonical and Standard Forms . . . 4

3 Karnaugh Map (K-Map) Simplification . . . 5
- 3.1 Grouping Principles and Implicants . . . 5
- 3.2 Static Hazards in Combinational Logic . . . 5

4 Binary Codes and Code Converters . . . 6
- 4.1 Standard Binary Codes . . . 6
- 4.2 Binary to Gray and Gray to Binary Code Conversion . . . 6

5 Combinational Arithmetic Circuits . . . 6
- 5.1 Half Adder, Full Adder, and Subtractors . . . 6
- 5.2 Ripple Carry Adder vs. Carry Lookahead Adder (CLA) . . . 7
- 5.3 BCD Adder . . . 7

6 MSI Combinational Logic Devices . . . 7
- 6.1 Magnitude Comparators . . . 7
- 6.2 Multiplexers (Data Selectors) . . . 8
- 6.3 Encoders and Decoders . . . 8
- 6.4 Barrel Shifters and Arithmetic Logic Units (ALUs) . . . 8

7 Quick Revision Page . . . 9

1. Number Systems and Binary Arithmetic

Digital systems process discrete electrical signals representing numerical data in positional number systems. A number N in a base (radix) r is represented as:

$$(N)_r = \sum_{i=-m}^{n-1} d_i \cdot r^i = d_{n-1}r^{n-1} + \cdots + d_1 r^1 + d_0 r^0 + d_{-1}r^{-1} + \cdots + d_{-m}r^{-m} \tag{1}$$

where $0 \leq d_i \leq (r-1)$, n is the number of integer digits, and m is the number of fractional digits.

1.1 Radix Conversions

- **Base-r to Decimal:** Expand the number into polynomials of powers of r.
- **Decimal to Base-r:** Divide the integer portion successively by r (collecting remainders from LSB to MSB); multiply the fractional portion successively by r (collecting integer carries from MSB to LSB).
- **Binary, Octal, and Hexadecimal Conversions:** Because $8 = 2^3$ and $16 = 2^4$, conversion between binary and octal requires grouping bits into groups of 3; conversion between binary and hexadecimal requires grouping bits into groups of 4.

1.2 Representation of Signed Numbers

In an n-bit binary system, unsigned numbers range from 0 to $2^n - 1$. Signed numbers dedicate the most significant bit (MSB) as the sign bit: MSB $= 0 \implies$ Positive, MSB $= 1 \implies$ Negative.

© Biraj

Three Signed Number Representations (n-bit)

1. **Signed Magnitude:** Magnitude is stored in the lower $(n-1)$ bits.

$$\text{Range: } -(2^{n-1}-1) \leq N \leq +(2^{n-1}-1) \tag{2}$$

   Drawback: Two distinct representations for zero ($+0 = 0000_2$, $-0 = 1000_2$), complicating ALU hardware.

2. **1's Complement:** Negative numbers are formed by bitwise inverting all n bits of the positive magnitude.

$$\text{Range: } -(2^{n-1}-1) \leq N \leq +(2^{n-1}-1) \tag{3}$$

   Drawback: Dual representation for zero ($+0 = 0000_2$, $-0 = 1111_2$); requires an end-around carry addition.

3. **2's Complement (Universal Standard):** Negative numbers are formed by adding 1 to the 1's complement ($N_{2's} = 2^n - |N|$).

$$\text{Range: } -2^{n-1} \leq N \leq +(2^{n-1}-1) \tag{4}$$

   Advantages: Single, unique representation for zero (0000_2); arithmetic subtraction is performed using standard binary addition without end-around carry; extra negative value can be represented.

1.3 2's Complement Arithmetic and Overflow Detection

To compute $A-B$, the digital processor computes $A+(\text{2's complement of } B)$. An end-carry produced out of the MSB is simply discarded.

Arithmetic Overflow Rule in 2's Complement

Overflow occurs when an addition of two numbers with identical signs produces an invalid result with an opposite sign. Overflow cannot occur when adding numbers with opposite signs.

$$V = C_n \oplus C_{n-1} \tag{5}$$

where C_n is the carry out of the sign bit (MSB), and C_{n-1} is the carry into the sign bit. If $V = 1$, arithmetic overflow has occurred, indicating the result exceeds the allowable n-bit range.

2. Boolean Algebra and Logic Simplification

Boolean algebra is an algebraic structure defined on a set of elements $B = \{0, 1\}$ with two binary operators ($+$ OR, $\cdot$ AND) and one unary operator ($'$ NOT).

© Biraj

2.1 Fundamental Laws of Boolean Algebra

Law / Property	AND Form	OR Form
Identity Law	$A \cdot 1 = A$	$A + 0 = A$
Null (Annulment) Law	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotent Law	$A \cdot A = A$	$A + A = A$
Complement Law	$A \cdot A' = 0$	$A + A' = 1$
Involution (Double Negation)	$(A')' = A$	
Commutative Law	$A \cdot B = B \cdot A$	$A + B = B + A$
Associative Law	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$
Distributive Law	$A \cdot (B + C) = A \cdot B + A \cdot C$	$A + (B \cdot C) = (A + B) \cdot (A + C)$
Absorption Law	$A \cdot (A + B) = A$	$A + A \cdot B = A$
Consensus Theorem	$(A + B)(A' + C)(B + C) = (A + B)(A' + C)$	$AB + A'C + BC = AB + A'C$

2.2 De Morgan's Theorems and the Principle of Duality

De Morgan's Theorems

$$\overline{A \cdot B \cdot C \ldots} = \overline{A} + \overline{B} + \overline{C} + \ldots \quad \text{(NAND gate is equivalent to a Bubbled-OR gate)} \tag{6}$$

$$\overline{A + B + C + \ldots} = \overline{A} \cdot \overline{B} \cdot \overline{C} \ldots \quad \text{(NOR gate is equivalent to a Bubbled-AND gate)} \tag{7}$$

Principle of Duality: Any true Boolean identity remains valid if:

a. Every $+$ (OR) operator is replaced with $\cdot$ (AND), and vice versa.

b. Every 0 is replaced with 1, and vice versa.

c. Literal variables remain uncomplemented.

2.3 Canonical and Standard Forms

- **Minterm (m_i):** A product (AND) of all n literals in which each variable appears exactly once in true or complemented form. A minterm equals 1 for exactly one input combination.
- **Maxterm (M_i):** A sum (OR) of all n literals in which each variable appears exactly once. A maxterm equals 0 for exactly one input combination.
- **Complementary Duality:**

$$M_i = \overline{m_i}, \qquad f(A, B, C) = \sum m(1, 4, 6, 7) = \prod M(0, 2, 3, 5) \tag{8}$$

© Biraj

- **Sum of Products (SOP):** Canonical form summing all minterms where the function is 1.
- **Product of Sums (POS):** Canonical form multiplying all maxterms where the function is 0.

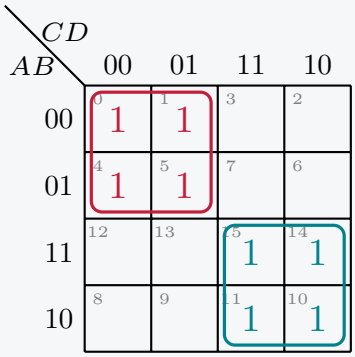
3. Karnaugh Map (K-Map) Simplification

The Karnaugh Map is a graphical truth-table layout where adjacent squares differ in only a single bit position (Gray code ordering: $00, 01, 11, 10$).

3.1 Grouping Principles and Implicants

1. Adjacent squares containing 1s are combined into groups of sizes that are powers of two $(1, 2, 4, 8, 16)$.
2. Grouping 2^k adjacent cells eliminates k variables.
3. **Implicant:** Any single minterm or combination of minterms that can be grouped.
4. **Prime Implicant (PI):** A rectangle of 1s that cannot be merged with any larger group.
5. **Essential Prime Implicant (EPI):** A prime implicant that covers at least one minterm (1) not covered by any other prime implicant. All EPIs must be included in the minimal Boolean expression.
6. **Don't Care Conditions (d or X):** Input combinations that can never occur in practice. They can be treated as 1s if they help enlarge a group, or as 0s if they do not.

4-Variable Karnaugh Map Cell Topology and Adjacency Structure



Quad 1 (EPI): $A'C'$

Quad 2 (EPI): AC

3.2 Static Hazards in Combinational Logic

A **static hazard** is a transient unwanted glitch ($0 \to 1 \to 0$ for static-0 or $1 \to 0 \to 1$ for static-1) occurring at the output due to unequal propagation delays along converging signal paths.

- **Cause:** A single variable changes state and transitions between two adjacent K-map prime implicant loops without an overlapping product term.
- **Elimination:** Add redundant consensus prime implicants to bridge adjacent loops in the K-map.

© Biraj

4. Binary Codes and Code Converters

Digital systems employ specialized binary codes tailored for arithmetic, display, communication, and mechanical position sensing:

4.1 Standard Binary Codes

- **BCD (Binary Coded Decimal / 8421):** Each decimal digit (0–9) is encoded as a 4-bit binary group (0000 to 1001). The six combinations 1010_2 to 1111_2 are invalid.
- **Excess-3 Code:** Unweighted, self-complementing code formed by adding 0011_2 $(+3)$ to the 8421 BCD code. The 1's complement of an Excess-3 number represents the 9's complement of its decimal equivalent.
- **Gray Code (Reflected Binary Unit-Distance Code):** Successive code words differ in only a single bit position ($\Delta = 1$), eliminating spurious switching errors in optical shaft encoders.

4.2 Binary to Gray and Gray to Binary Code Conversion

Code Conversion Logic

Binary to Gray conversion ($B = b_{n-1} \dots b_0 \rightarrow G = g_{n-1} \dots g_0$):

$$g_{n-1} = b_{n-1}, \qquad g_i = b_{i+1} \oplus b_i \quad (\text{for } i = 0, 1, \dots, n-2) \tag{9}$$

Gray to Binary conversion ($G = g_{n-1} \dots g_0 \rightarrow B = b_{n-1} \dots b_0$):

$$b_{n-1} = g_{n-1}, \qquad b_i = b_{i+1} \oplus g_i \quad (\text{for } i = 0, 1, \dots, n-2) \tag{10}$$

5. Combinational Arithmetic Circuits

5.1 Half Adder, Full Adder, and Subtractors

Circuit	Boolean Sum / Difference (S, D)	Carry / Borrow (C, B)
Half Adder (HA)	$S = A \oplus B$	$C = A \cdot B$
Half Subtractor (HS)	$D = A \oplus B$	$B_{\text{out}} = A' \cdot B$
Full Adder (FA)	$S = A \oplus B \oplus C_{\text{in}}$	$C_{\text{out}} = AB + (A \oplus B)C_{\text{in}} = AB + BC_{\text{in}} + AC_{\text{in}}$
Full Subtractor (FS)	$D = A \oplus B \oplus B_{\text{in}}$	$B_{\text{out}} = A'B + (A \oplus B)'B_{\text{in}} = A'B + A'B_{\text{in}} + BB_{\text{in}}$

© Biraj

5.2 Ripple Carry Adder vs. Carry Lookahead Adder (CLA)

An n-bit **Ripple Carry Adder (RCA)** cascades n Full Adders. The carry out of stage i ripples to the carry input of stage $i+1$. Total worst-case propagation delay is linear:

$$t_{\text{RCA}} = 2t_{\text{gate}} + (n-1)2t_{\text{gate}} = 2n\,t_{\text{gate}} \tag{11}$$

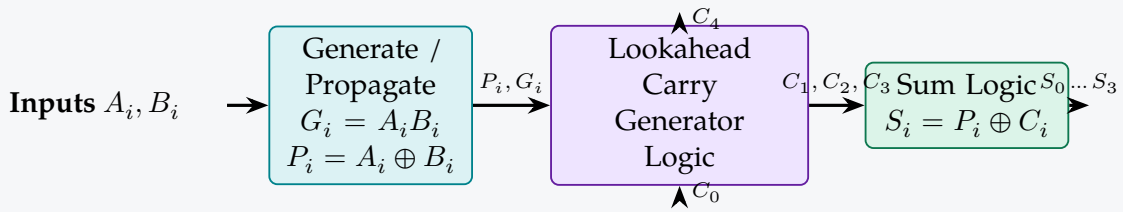
To eliminate rippling delay, the **Carry Lookahead Adder (CLA)** precomputes all stage carries in parallel using Carry Generate (G_i) and Carry Propagate (P_i) functions:

$$G_i = A_i \cdot B_i, \qquad P_i = A_i \oplus B_i \tag{12}$$

Expanding the stage carry expressions recursively:

$$C_1 = G_0 + P_0C_0 \tag{13}$$
$$C_2 = G_1 + P_1C_1 = G_1 + P_1G_0 + P_1P_0C_0 \tag{14}$$
$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0 \tag{15}$$
$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0 \tag{16}$$

Carry Lookahead Generator Architecture (4-bit CLA)



Delay Comparison: All carries $C_1, \dots, C_4$ are generated in only $2t_{\text{gate}}$ delays, and the final sums in $4t_{\text{gate}}$ delays, completely independent of word length n.

5.3 BCD Adder

Adding two 4-bit BCD digits produces a sum $S_3S_2S_1S_0$ and carry K. When the sum exceeds 9 (1001_2) or a carry $K = 1$ is generated, the result is an invalid BCD digit. Correction is achieved by adding 6 (0110_2):

$$\text{Correction Flag } C_{\text{out}} = K + S_3S_2 + S_3S_1 \tag{17}$$

6. MSI Combinational Logic Devices

6.1 Magnitude Comparators

A magnitude comparator compares two n-bit binary numbers $A = a_{n-1} \dots a_0$ and $B = b_{n-1} \dots b_0$ and asserts one of three output lines: $A > B$, $A = B$, or $A < B$.

- Equality condition: $A = B \implies x_i = (a_i \oplus b_i)'$ for all i, so $(A = B) = x_{n-1}x_{n-2} \dots x_0$.
- Greater-than condition:

$$(A > B) = a_{n-1}b'_{n-1} + x_{n-1}a_{n-2}b'_{n-2} + \cdots + x_{n-1} \dots x_1a_0b'_0 \tag{18}$$

- IC 7485 is a 4-bit magnitude comparator equipped with cascading inputs $(I_{A>B}, I_{A=B}, I_{A<B})$ enabling expansion to compare 8-bit, 16-bit, or larger words.

© Biraj

6.2 Multiplexers (Data Selectors)

A multiplexer routes one of 2^n data input channels to a single output line under the control of n selection lines:

$$Y = \sum_{k=0}^{2^n-1} m_k \cdot D_k \tag{19}$$

where m_k is the minterm formed by the n select lines $S_{n-1} \dots S_0$.

- **Universal Logic Implementation:** Any Boolean function of n variables can be implemented directly using a 2^{n-1}-to-1 MUX by connecting $(n-1)$ variables to select lines and using the remaining variable $(0, 1, X, X')$ on the data inputs.

6.3 Encoders and Decoders

- **Decoders (n-to-2^n):** Converts an n-bit binary input into one of 2^n active output lines (minterm generator). IC 74138 is a standard 3-to-8 decoder with active-low outputs.
- **Priority Encoders (2^n-to-n):** If two or more inputs are simultaneously asserted, the encoder generates the binary code corresponding to the input with the highest assigned priority (e.g., IC 74148 8-to-3 priority encoder).

6.4 Barrel Shifters and Arithmetic Logic Units (ALUs)

- **Barrel Shifter:** A specialized digital crossbar circuit constructed from multiplexer arrays capable of shifting or rotating an n-bit data word by an arbitrary number of bit positions (0 to $n-1$) in a **single clock cycle**, avoiding sequential shift register clock cycles.
- **Arithmetic Logic Unit (IC 74181):** A 4-bit parallel multifunction processor element. Under the command of 4 function select inputs ($S_3S_2S_1S_0$) and a mode control bit M ($M = 1 \implies$ Logic functions; $M = 0 \implies$ Arithmetic functions with carry lookahead), it can perform all 16 logic operations and 16 arithmetic operations.

© Biraj

7. Quick Revision Page

Key Formulas and Identities at a Glance

- **Signed 2's Complement Range:** $-2^{n-1} \leq N \leq 2^{n-1} - 1$
- **2's Complement Overflow Condition:** $V = C_n \oplus C_{n-1}$
- **De Morgan's Laws:** $\overline{A \cdot B} = \overline{A} + \overline{B}, \quad \overline{A + B} = \overline{A} \cdot \overline{B}$
- **Binary-to-Gray Conversion:** $g_{n-1} = b_{n-1}, \quad g_i = b_{i+1} \oplus b_i$
- **Full Adder Sum & Carry:** $S = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = AB + (A \oplus B)C_{\text{in}}$
- **CLA Generate & Propagate:** $G_i = A_i B_i, \quad P_i = A_i \oplus B_i, \quad C_{i+1} = G_i + P_i C_i$
- **BCD Adder Correction:** $C_{\text{out}} = K + S_3 S_2 + S_3 S_1$
- **Multiplexer Output Equation:** $Y = \sum_{k=0}^{2^n-1} m_k D_k$

One-Line Definitions

- **Minterm:** A canonical product term of n variables that evaluates to logic 1 for exactly one input combination.
- **Essential Prime Implicant (EPI):** A prime implicant rectangle containing at least one minterm that is not covered by any other prime implicant.
- **Static-1 Hazard:** A transient output glitch to 0 occurring when an input change should have maintained a constant 1 output.
- **Gray Code:** An unweighted, unit-distance binary code in which adjacent numbers differ by exactly one bit.
- **Priority Encoder:** An encoder that outputs the binary address of only the highest-priority active input line.
- **Barrel Shifter:** A combinational logic circuit that shifts or rotates an n-bit word by any specified number of bits in a single clock cycle.

Frequently Asked / Viva Questions

Q1. Why is 2's complement universally preferred over 1's complement in digital ALUs?

2's complement provides a single unique representation for zero (0000_2), eliminates the complex end-around carry addition required in 1's complement, and simplifies hardware subtraction into simple addition.

Q2. How do you detect arithmetic overflow in 2's complement addition?

Overflow occurs when adding two numbers of identical sign results in a sum with the opposite sign. In hardware, it is detected when the carry into the sign bit differs from the carry out of the sign bit ($V = C_n \oplus C_{n-1} = 1$).

© Biraj

Frequently Asked / Viva Questions (contd.)

Q3. What is the difference between Prime Implicants and Essential Prime Implicants in a K-map?

A Prime Implicant is a maximal group of adjacent 1s that cannot be doubled in size. An Essential Prime Implicant is a prime implicant that covers at least one 1 that is not covered by any other prime implicant.

Q4. Explain the cause and remedy of a static-1 hazard.

It is caused by unequal propagation delays when a variable and its complement change simultaneously across two separate K-map loops. It is remedied by adding an extra overlapping consensus prime implicant to cover the transition.

Q5. Why is Gray code extensively used in optical shaft encoders?

Because only a single bit changes between any two consecutive values ($\Delta = 1$), avoiding erroneous transient readings caused by mechanical switch misalignment.

Q6. Why is a Carry Lookahead Adder (CLA) faster than a Ripple Carry Adder (RCA)?

In an RCA, carry must propagate sequentially through all n stages ($t \propto n$). In a CLA, carries are computed simultaneously in two gate delays using carry generate (G_i) and propagate (P_i) logic ($t = O(1)$).

Q7. What is the correction condition in a BCD adder?

When the binary sum of two BCD digits exceeds 9 (1001_2) or generates an output carry $K = 1$, the correction logic $C = K + S_3S_2 + S_3S_1$ adds $+6$ (0110_2) to skip the 6 invalid 4-bit states.

Q8. How can an n-variable Boolean function be implemented using an 2^{n-1}-to-1 MUX?

Connect $(n-1)$ variables to the select lines, and connect the data input lines to $0, 1$, or the remaining variable X or X' derived from a truth table partition.

Q9. What is a Priority Encoder and where is it used?

It is an encoder that produces the binary code corresponding to the highest-order active input when multiple inputs are asserted. It is widely used in microprocessor interrupt request handling.

Q10. What is the function of the mode control bit M in the 74181 ALU?

$M = 1$ disables internal carry propagation, configuring the ALU for 16 logic operations. $M = 0$ enables carries, configuring the ALU for 16 arithmetic operations.

Q11. Differentiate between canonical and standard Boolean expressions.

In canonical form (canonical SOP/POS), every individual product/sum term must contain all variables of the function. In standard form, terms may contain any subset of variables.

© Biraj

Frequently Asked / Viva Questions (contd.)

Q12. What is a barrel shifter and why is it preferred over a shift register in microprocessors?

A barrel shifter uses multiplexer matrices to shift or rotate an n-bit word by k positions in a single clock cycle, whereas an ordinary shift register requires k separate clock cycles.

Comparison of Combinational MSI Building Blocks

Device	Input / Output Ratio	Control Signals	Primary Application
Multiplexer (MUX)	2^n Data Inputs $\rightarrow$ 1 Output	n Select Lines, Enable	Data routing, parallel-to-serial conversion, Boolean function synthesis
Demultiplexer (DEMUX)	1 Data Input $\rightarrow$ 2^n Outputs	n Select Lines, Enable	Serial-to-parallel distribution, memory addressing
Decoder	n Inputs $\rightarrow$ 2^n Outputs	Enable / Chip Select	Memory address decoding, 7-segment displays, minterm generation
Priority Encoder	2^n Inputs $\rightarrow$ n Outputs	Enable, Group Select	Interrupt arbitration, ADC flash encoding
Magnitude Comparator	$2 \times n$ Bit Inputs $\rightarrow$ 3 Outputs	Cascading Inputs $(>, =, <)$	Arithmetic sorting, threshold checking, address limit checks

© Biraj